

# Software Construction & Development (Comp-370)



## UNIVERSITY ONLINE ELECTION VOTING SYSTEM

### Project Documentation

#### Submitted By:

**Atiqad Hayat**  
**B23F0527SE047**

**Muhammad Bilal**  
**Sami**  
**B23S0295SE014**

#### Submitted To:

**Malik Nabeel Ahmed Awan**

**Date: 3 May 2026**

---

# Table of Contents

Abstract

Table of Contents

|                                                     |    |
|-----------------------------------------------------|----|
| Abstract .....                                      | 4  |
| 1. Introduction .....                               | 4  |
| 2. Problem Statement .....                          | 4  |
| 3. Objectives .....                                 | 5  |
| 4. Scope of the Project .....                       | 5  |
| 5. Technologies Used .....                          | 6  |
| 6. System Architecture .....                        | 6  |
| 7. System Modules .....                             | 7  |
| 7.1 Authentication and User Management Module ..... | 7  |
| 7.2 Election Management Module .....                | 7  |
| 7.3 Candidate Management Module .....               | 8  |
| 7.4 Voting Module .....                             | 8  |
| 7.5 Result and Winner Module .....                  | 8  |
| 8. System Workflow .....                            | 8  |
| 9. User Roles and Permissions .....                 | 9  |
| 10. REST API Communication .....                    | 9  |
| 11. Database Design .....                           | 9  |
| 12. API Endpoints .....                             | 10 |
| 13. Backend Implementation Details .....            | 10 |
| 14. Frontend Implementation Details .....           | 11 |

|                                                 |        |
|-------------------------------------------------|--------|
| 15. Security Implementation .....               | 12     |
| 16. Design Patterns and Principles .....        | 12     |
| 17. Testing and Results .....                   | 13     |
| 18. Team Collaboration .....                    | 15     |
| 19. Challenges Faced .....                      | 16     |
| 20. Sustainable Development Goals Mapping ..... | 16     |
| 21. Conclusion .....                            | 16     |
| 22. Future Enhancements .....                   | 16     |
| 23. References .....                            | 17     |
| <br>Appendix: Diagrams and Screenshots .....    | <br>18 |
| Architecture Diagram .....                      | 18     |
| Sequence Diagram .....                          | 19     |
| Activity Diagram .....                          | 20     |
| Testing Results and System Verification .....   | 21     |
| Student Voting Page Verification .....          | 22     |
| Candidate Management Testing .....              | 22     |
| Election Management Testing .....               | 23     |
| Result and Winner Page Verification .....       | 23     |

## Abstract

The University Online Election Voting System is a web-based software application designed to digitalize the election process within a university environment. The system provides a structured platform where administrators can manage elections and candidates, while students can securely access the system and cast their votes through an online interface.

The project is implemented using Spring Boot for backend development and MySQL for persistent database storage. The backend exposes REST APIs for communication between the client-side interface and the server-side business logic. The system follows a layered architecture that separates controllers, services, repositories, entities, and database responsibilities.

The main purpose of the system is to reduce manual work in university elections, improve data accuracy, prevent duplicate voting, and provide automated result calculation. The project demonstrates practical use of RESTful API development, database integration, role-based access, CRUD operations, and modular software design.

## 1. Introduction

University elections are important for selecting student representatives and managing student leadership roles. In many institutions, this process is still handled manually through paper-based voting, manual candidate records, and manual result counting. These traditional methods are time-consuming and can lead to errors, delays, and difficulties in record management.

The University Online Election Voting System provides a digital solution for managing the election process. It allows the administration side to manage elections and candidates, while students can participate in voting using a secure login-based system. The system improves efficiency by storing records in a centralized database and calculating results automatically.

This project has been developed as a full-stack university project with a Spring Boot backend, MySQL database, REST API communication, and a frontend interface for user interaction. The system is designed with a modular structure so that future features such as advanced audit logs, OTP verification, deployment, and result export can be added easily.

## 2. Problem Statement

Manual election systems in universities require significant administrative effort. Candidate registration, voter verification, vote casting, vote counting, and result announcement are often performed manually. This creates delays and increases the possibility of human error.

- Manual vote counting can take more time and may produce inaccurate results.
- Student eligibility verification is difficult when records are not centralized.
- Duplicate voting is harder to prevent without automated validation.
- Candidate and election management becomes difficult when handled through paper records.
- Result generation is delayed because votes must be counted manually.
- The absence of a digital record makes future verification and auditing difficult.

The proposed system solves these issues by providing an online platform where election records, candidate data, student voting status, and results are managed digitally.

### 3. Objectives

The main objectives of the project are:

- To design and implement an online voting platform for university elections.
- To provide role-based access for Admin and Student users.
- To allow administrators to create and manage elections.
- To allow administrators to add, update, and remove candidates.
- To allow students to vote digitally in active elections.
- To prevent a student from voting more than once in the same election.
- To store all records in a MySQL database.
- To calculate election results automatically.
- To display winner information based on vote count.
- To provide a clean and usable frontend interface for both roles.

### 4. Scope of the Project

The scope of this project includes the development of a university election management and voting platform. The system focuses on managing elections, candidates, users, votes, and results through a structured backend and interactive frontend.

Implemented Scope

- Spring Boot backend with REST API endpoints.
- MySQL database integration.
- User management and login flow.
- Role-based access for Admin and Student users.
- Election management module.
- Candidate management module.
- Voting module with duplicate vote prevention.
- Automatic result and winner calculation.
- Frontend pages for login, dashboards, elections, candidates, voting, results, and winner display.
- Postman-based backend API testing.

Out of Scope / Not Implemented

- Cloud deployment is not included in the current version.
- OTP or email verification is not implemented.
- Biometric verification is not implemented.
- Advanced analytics and PDF result export are future enhancements.

## 5. Technologies Used

| Technology         | Purpose                                                                |
|--------------------|------------------------------------------------------------------------|
| Java               | Main programming language used for backend business logic.             |
| Spring Boot        | Backend framework used to develop REST APIs and application layers.    |
| Spring Data JPA    | Used for database interaction and repository abstraction.              |
| MySQL              | Relational database used for persistent storage.                       |
| REST APIs          | Used for communication between frontend and backend.                   |
| Frontend Framework | Used to build user interface pages and connect them with backend APIs. |
| Maven              | Used for dependency management and project build configuration.        |
| IntelliJ IDEA      | Development environment used for backend implementation.               |
| Postman            | Used for testing REST API endpoints.                                   |
| GitHub             | Used for project version control, submission, and collaboration.       |

## 6. System Architecture

The system follows a layered architecture. Each layer has a separate responsibility, which makes the project easier to understand, test, maintain, and extend.

- Frontend Layer: Provides user interface pages for Admin and Student users.
- Controller Layer: Receives HTTP requests and maps them to backend functions.
- Service Layer: Contains business logic such as validation, voting rules, and result calculation.
- Repository Layer: Handles database operations using Spring Data JPA.
- Database Layer: Stores users, elections, candidates, votes, and result-related data in MySQL.

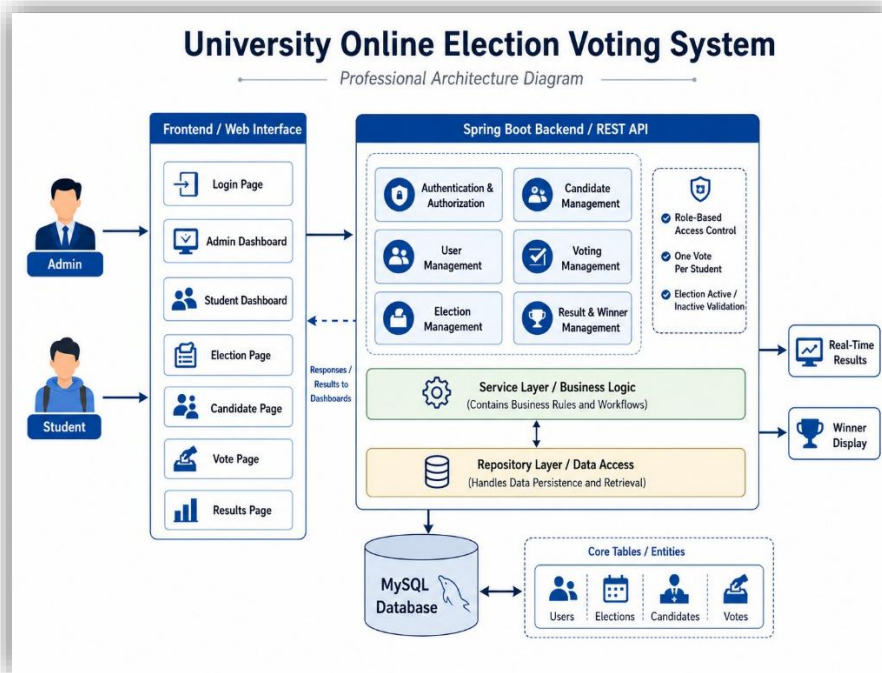


Figure 1: Layered architecture of the University Online Election Voting System.

## 7. System Modules

### 7.1 Authentication and User Management Module

The Authentication and User Management Module controls user access to the system. Users are stored with essential details such as full name, registration number, email, password, and role. The role value identifies whether the user is an Admin or Student.

- Registers or stores users in the database.
- Allows login using valid credentials.
- Differentiates users based on Admin and Student roles.
- Supports API-based user operations such as add user and get all users.

### 7.2 Election Management Module

The Election Management Module allows the Admin to manage elections. It stores election details and supports the election lifecycle. An election can be used to group candidates and votes under one voting event.

- Create election records.
- Update election information.
- Delete election records where required.
- Control active or inactive election state if implemented.
- Connect candidates and votes with the relevant election.

## 7.3 Candidate Management Module

The Candidate Management Module allows administrators to manage candidate records for elections. Candidate information may include candidate name, department, program, contact number, biography, election reference, and candidate photo where supported by the implementation.

- Add candidates to an election.
- Update candidate information.
- Delete candidate records.
- Display candidates for voting and result pages.

## 7.4 Voting Module

The Voting Module handles the main voting operation. After a student logs in, the system allows the student to view candidates and cast a vote. The system validates whether the student has already voted in the selected election before saving the vote.

- Allows a logged-in student to cast a vote.
- Checks duplicate voting before saving a vote.
- Stores vote records in the database.
- Links each vote with a student, candidate, and election.

## 7.5 Result and Winner Module

The Result and Winner Module calculates vote counts and displays results. It reads saved votes from the database, groups them by candidate, and identifies the candidate with the highest number of votes as the winner.

- Counts votes for each candidate.
- Displays results in the result dashboard/page.
- Identifies and displays the winning candidate.
- Supports transparent result viewing after voting.

# 8. System Workflow

The system workflow explains how users interact with the application and how data moves from frontend pages to backend APIs and then to the database.

1. Admin logs in using valid credentials.
2. The system verifies the admin role and opens the admin dashboard.
3. Admin creates or manages elections.
4. Admin adds, updates, or removes candidates for an election.
5. Student logs in using valid credentials.
6. The system verifies the student role and opens the student dashboard.
7. Student views active election and candidate information.
8. Student selects the vote button to cast a vote.
9. The backend checks whether the student has already voted in that election.

10. If eligible, the vote is saved in the database.
11. The result module counts saved votes.
12. The winner page displays the candidate with the highest vote count.

## 9. User Roles and Permissions

| Role    | Description                                              | Allowed Actions                                                                        | Restricted Actions                                                                |
|---------|----------------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Admin   | Administrative user responsible for election management. | Manage elections, manage candidates, view votes, view results, view winner dashboard.  | Cannot cast student vote unless specifically implemented as student role.         |
| Student | Registered voter who participates in an election.        | Login, view active elections, view candidates, cast one vote, view results if allowed. | Cannot create elections, cannot manage candidates, cannot access admin dashboard. |

## 10. REST API Communication

The backend communicates through REST APIs. The frontend sends HTTP requests to backend controllers, and the backend returns structured responses. This approach keeps frontend and backend communication clean and modular.

### Typical Communication Flow

13. Frontend sends an HTTP request to an API endpoint.
14. Controller receives the request.
15. Controller calls the service layer.
16. Service layer applies business logic and validation.
17. Repository layer performs database operations.
18. Response is returned to the frontend in structured form.

## 11. Database Design

The project uses MySQL as the database. The database stores all important records required for the election process.

| Entity    | Purpose                     | Important Fields                                                 | Relationship                                          |
|-----------|-----------------------------|------------------------------------------------------------------|-------------------------------------------------------|
| User      | Stores users of the system. | id, fullName, regNo, email, password, role                       | Primary entity for login and role-based access.       |
| Election  | Stores election details.    | id, title/name, description, start time, end time, active status | One election can have many candidates and many votes. |
| Candidate | Stores candidate details.   | id, name, department, program, contact, bio, photo, election_id  | Candidate belongs to one election.                    |
| Vote      | Stores voting transaction.  | id, user_id/student_id, candidate_id, election_id                | Vote links student, candidate, and election.          |

### Entity Relationship Explanation

- One election can contain many candidates.

- One candidate belongs to one election.
- One student can cast only one vote in a specific election.
- One vote belongs to one student, one candidate, and one election.
- Results are calculated by counting votes against candidates.

## 12. API Endpoints

The following API documentation represents the major endpoint groups used in the system. Exact endpoint naming may vary according to the controller mapping used in the project source code.

| Module        | Method | Endpoint         | Description                               | Access                 |
|---------------|--------|------------------|-------------------------------------------|------------------------|
| User API      | POST   | /users/add       | Creates a new user record.                | Admin / Public testing |
| User API      | GET    | /users/all       | Returns all users stored in the database. | Admin / Testing        |
| Election API  | POST   | /elections       | Creates an election record.               | Admin                  |
| Election API  | GET    | /elections       | Returns election records.                 | Admin / Student        |
| Election API  | PUT    | /elections/{id}  | Updates election information.             | Admin                  |
| Election API  | DELETE | /elections/{id}  | Deletes an election record.               | Admin                  |
| Candidate API | POST   | /candidates      | Adds a candidate.                         | Admin                  |
| Candidate API | GET    | /candidates      | Returns candidate records.                | Admin / Student        |
| Candidate API | PUT    | /candidates/{id} | Updates candidate information.            | Admin                  |
| Candidate API | DELETE | /candidates/{id} | Deletes candidate record.                 | Admin                  |
| Vote API      | POST   | /votes           | Stores a student vote after validation.   | Student                |
| Result API    | GET    | /results         | Returns vote counts/results.              | Admin / Student        |
| Winner API    | GET    | /winner          | Returns winning candidate details.        | Admin / Student        |

### Sample Vote Request Body

```
{
  "studentId": 1,
  "candidateId": 3,
  "electionId": 2
}
```

### Sample Result Response

```
{
  "candidateName": "Candidate A",
  "totalVotes": 25,
  "electionName": "University Election 2026"
}
```

## 13. Backend Implementation Details

The backend implementation is organized into multiple layers. This structure keeps controller code clean and places business logic inside service classes.

## Controller Layer

Controllers expose REST endpoints and receive frontend requests. They do not directly handle database operations; instead, they delegate work to service classes.

## Service Layer

The service layer contains application logic such as user verification, duplicate vote checking, election validation, candidate handling, result counting, and winner calculation.

## Repository Layer

Repositories use Spring Data JPA to perform database operations. They reduce the need for manual SQL queries and provide a clean abstraction over MySQL tables.

## Entity Layer

Entities represent database tables as Java classes. Each entity maps fields to database columns and defines relationships between records.

## Core Business Logic

- Login logic verifies user credentials and identifies role.
- Election logic manages election data and active/inactive status where implemented.
- Candidate logic handles candidate CRUD operations.
- Vote logic checks duplicate voting before saving a vote.
- Result logic counts votes candidate-wise.
- Winner logic identifies the candidate with the highest vote count.

# 14. Frontend Implementation Details

The frontend provides a role-based user interface for Admin and Student users. It connects with backend APIs to display data and perform system operations.

| Page              | Purpose                                                                     |
|-------------------|-----------------------------------------------------------------------------|
| Login Page        | Allows users to enter email and password and choose role-based access flow. |
| Admin Dashboard   | Provides access to election, candidate, result, and management pages.       |
| Student Dashboard | Shows voting-related options for logged-in students.                        |
| Election Page     | Displays or manages election information.                                   |
| Candidate Page    | Displays candidates and supports admin candidate management.                |
| Vote Page         | Allows students to cast a vote using a direct vote button.                  |
| Results Page      | Shows calculated vote results.                                              |
| Winner Page       | Displays the winning candidate based on highest vote count.                 |

## 15. Security Implementation

Security in this project focuses on controlled access, role separation, and duplicate vote prevention. The system uses login-based access and separates Admin and Student actions.

### Implemented Security Features

- Role-based access structure for Admin and Student users.
- Protected frontend pages so users cannot directly access restricted pages without login.
- Backend validation to prevent duplicate voting.
- Database relationships to maintain consistency between users, elections, candidates, and votes.
- Input validation where implemented through backend logic and forms.

### Security Limitations

- JWT token-based authentication is not confirmed in the current version.
- OTP/email verification is not implemented.
- Advanced audit trail is not implemented.
- Password encryption should be added if not already configured in the codebase.

## 16. Design Patterns and Principles

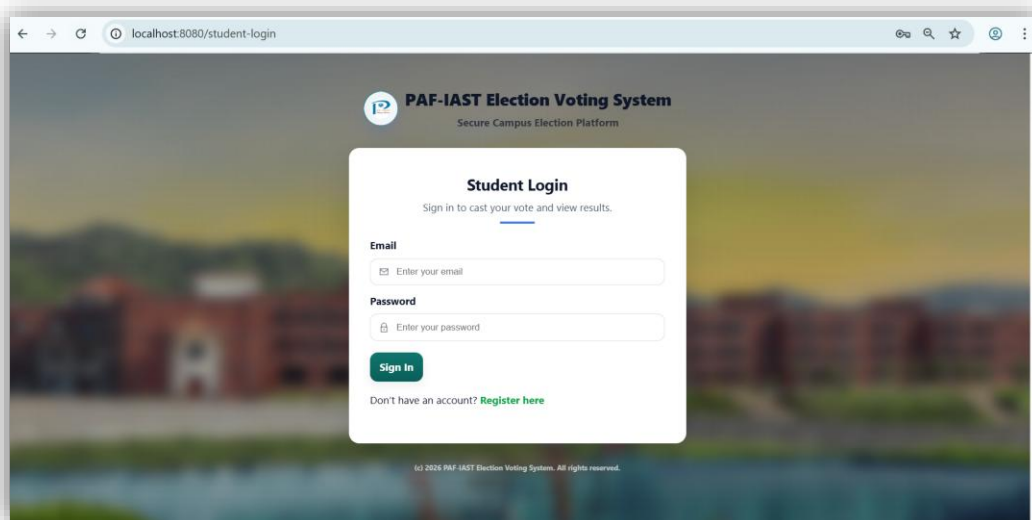
| Pattern / Principle             | Where Used                                                                                       | Benefit                                                               |
|---------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Layered Architecture            | Used across frontend, controller, service, repository, and database layers.                      | Improves maintainability and separates responsibilities.              |
| MVC Pattern                     | Controllers handle requests, models/entities represent data, and frontend views display results. | Supports clean separation between data, control logic, and interface. |
| Repository Pattern              | Repository interfaces manage database operations.                                                | Reduces direct dependency on SQL and simplifies data access.          |
| Service Layer Pattern           | Service classes contain business logic.                                                          | Keeps controllers clean and centralizes rules.                        |
| Dependency Injection            | Spring injects services and repositories where needed.                                           | Promotes loose coupling and easier testing.                           |
| Single Responsibility Principle | Classes are organized by responsibility such as controller, service, repository, and entity.     | Makes code easier to understand and modify.                           |
| Dependency Inversion Principle  | High-level components rely on abstractions such as repository interfaces.                        | Improves flexibility and modularity.                                  |

## 17. Testing and Results

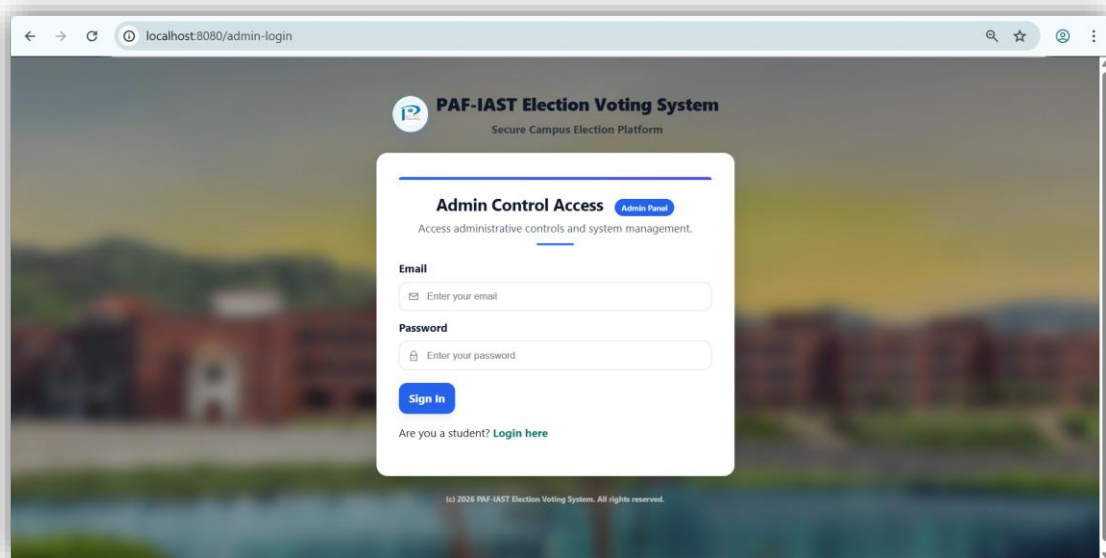
Testing was performed using Postman, browser testing, frontend flow checking, and database verification. The purpose of testing was to confirm that APIs, CRUD operations, voting validation, and result calculation work correctly.

| Test ID | Module         | Scenario                     | Input                        | Expected Result                   | Status |
|---------|----------------|------------------------------|------------------------------|-----------------------------------|--------|
| TC-01   | User           | Add new user                 | Valid user data              | User saved successfully           | Pass   |
| TC-02   | User           | Get all users                | GET request                  | All users returned                | Pass   |
| TC-03   | Authentication | Login with valid credentials | Email/password               | Dashboard opens according to role | Pass   |
| TC-04   | Election       | Create election              | Election data                | Election created                  | Pass   |
| TC-05   | Candidate      | Add candidate                | Candidate details            | Candidate saved                   | Pass   |
| TC-06   | Candidate      | Update candidate             | Updated details              | Candidate updated                 | Pass   |
| TC-07   | Voting         | Student casts vote           | Student, candidate, election | Vote saved                        | Pass   |
| TC-08   | Voting         | Duplicate vote attempt       | Same student and election    | System blocks duplicate vote      | Pass   |
| TC-09   | Result         | View results                 | Result request               | Vote counts displayed             | Pass   |
| TC-10   | Winner         | View winner                  | Winner request               | Winning candidate displayed       | Pass   |

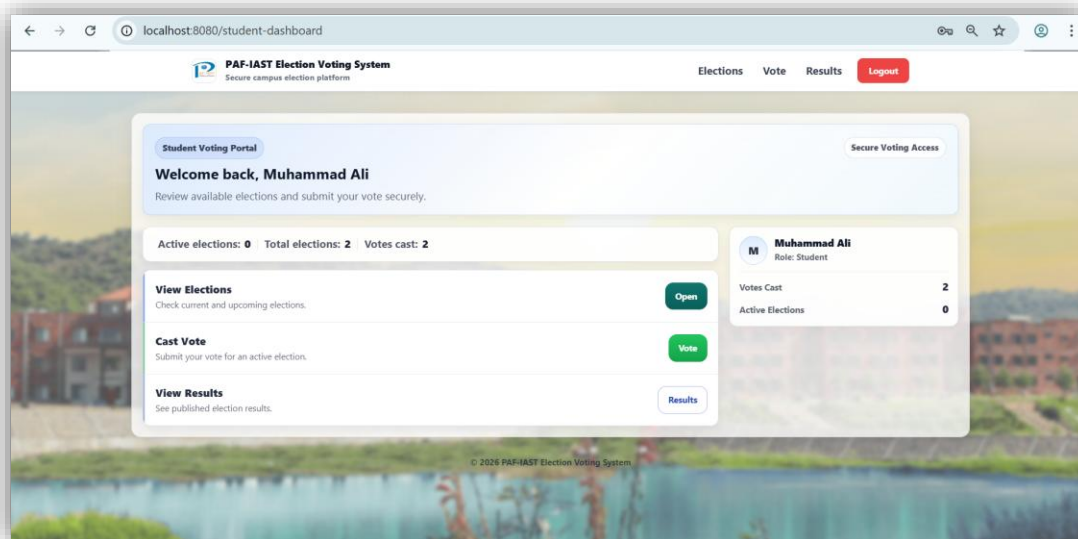
### Login Page



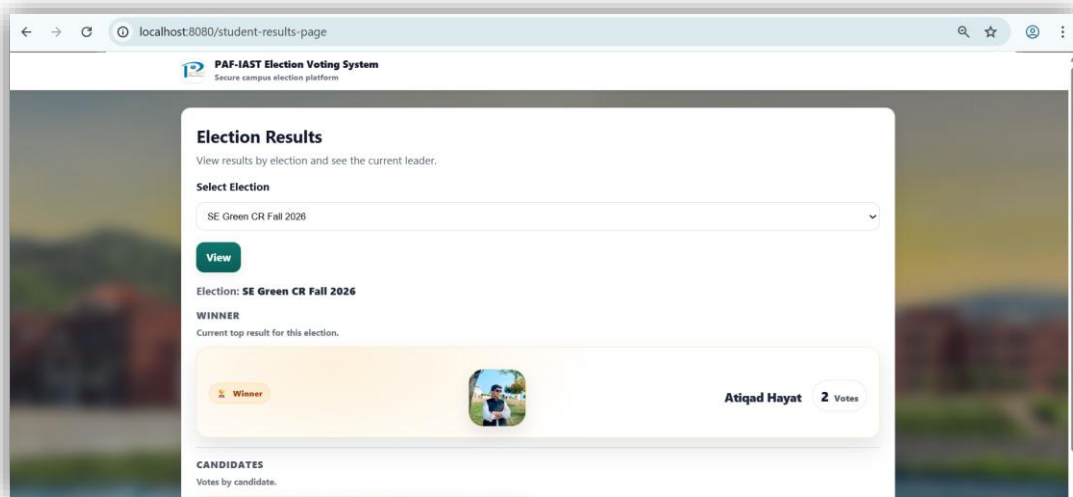
### Admin Dashboard



## Student Voting Page



## Results and Winner Page



## 18. Team Collaboration

This was a two-member project. The work was divided between backend development and frontend development, and the complete project was prepared for GitHub submission.

### Atiqad Hayat

- Backend design and implementation.
- Spring Boot API development.
- Database integration with MySQL.
- User, election, candidate, vote, and result module support.
- Postman API testing.
- GitHub repository setup and project submission preparation.

### Muhammad Bilal Sami

- Frontend design and implementation.
- UI pages for login, dashboards, elections, candidates, voting, results, and winner display.
- Frontend API integration.
- Documentation and diagram support.
- GitHub collaboration and project organization.

## 19. Challenges Faced

- Spring Boot package structure required careful organization so controllers, services, repositories, and entities could be detected properly.
- MySQL database configuration required correct database name, username, password, and connection URL.
- Role-based navigation required separating Admin and Student access.
- Duplicate vote prevention required validation before saving vote records.
- Frontend and backend integration required matching API endpoints and request formats.
- GitHub collaboration required careful commits, branch handling, and project structure management.

## 20. Sustainable Development Goals Mapping

This project can be connected with multiple Sustainable Development Goals because it promotes digital governance, transparency, and efficient institutional processes.

| SDG                                            | Mapping Explanation                                                                                                |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| SDG 4: Quality Education                       | The system supports academic institutions by providing a digital solution for student elections.                   |
| SDG 9: Industry, Innovation and Infrastructure | The project uses modern software engineering practices, REST APIs, database integration, and layered architecture. |
| SDG 16: Peace, Justice and Strong Institutions | Digital voting supports transparent, accountable, and organized institutional decision-making.                     |
| SDG 12: Responsible Consumption and Production | Online voting can reduce paper-based election materials and manual record keeping.                                 |

## 21. Conclusion

The University Online Election Voting System successfully provides a digital foundation for managing university elections. It supports role-based access, election management, candidate management, student voting, duplicate vote prevention, result calculation, and winner display.

The project demonstrates important software engineering concepts such as layered architecture, REST API development, database design, frontend-backend integration, and modular implementation. By replacing manual voting activities with a digital workflow, the system improves efficiency, accuracy, and transparency in the election process.

## 22. Future Enhancements

- Add OTP or email verification for stronger voter identity validation.
- Use JWT authentication for secure token-based access.
- Encrypt passwords using BCrypt.
- Deploy the system on a cloud platform.

- Add advanced audit logs for admin and voting activities.
- Export election results as PDF.
- Add real-time charts and analytics dashboard.
- Add email notifications for election announcements.
- Improve UI/UX with more responsive design.
- Add backup and recovery support for database safety.

## 23. References

- Spring Boot Documentation
- Spring Data JPA Documentation
- MySQL Documentation
- REST API Design Concepts
- Software Engineering Design Pattern Concepts
- GitHub Version Control Concepts

## Appendix: Diagrams and Screenshots

### Architecture Diagram

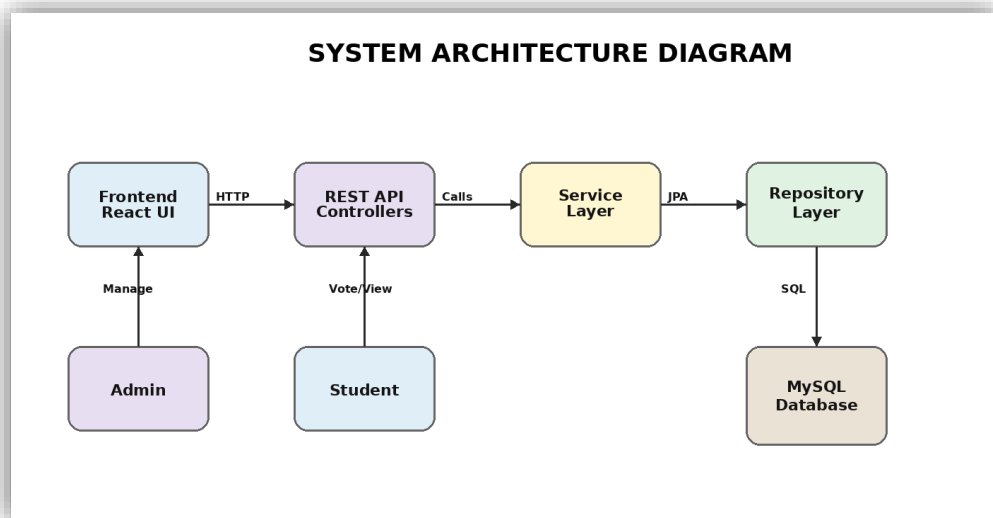


Figure 2: Complete architecture view showing frontend, backend layers, and MySQL database.

## Sequence Diagram

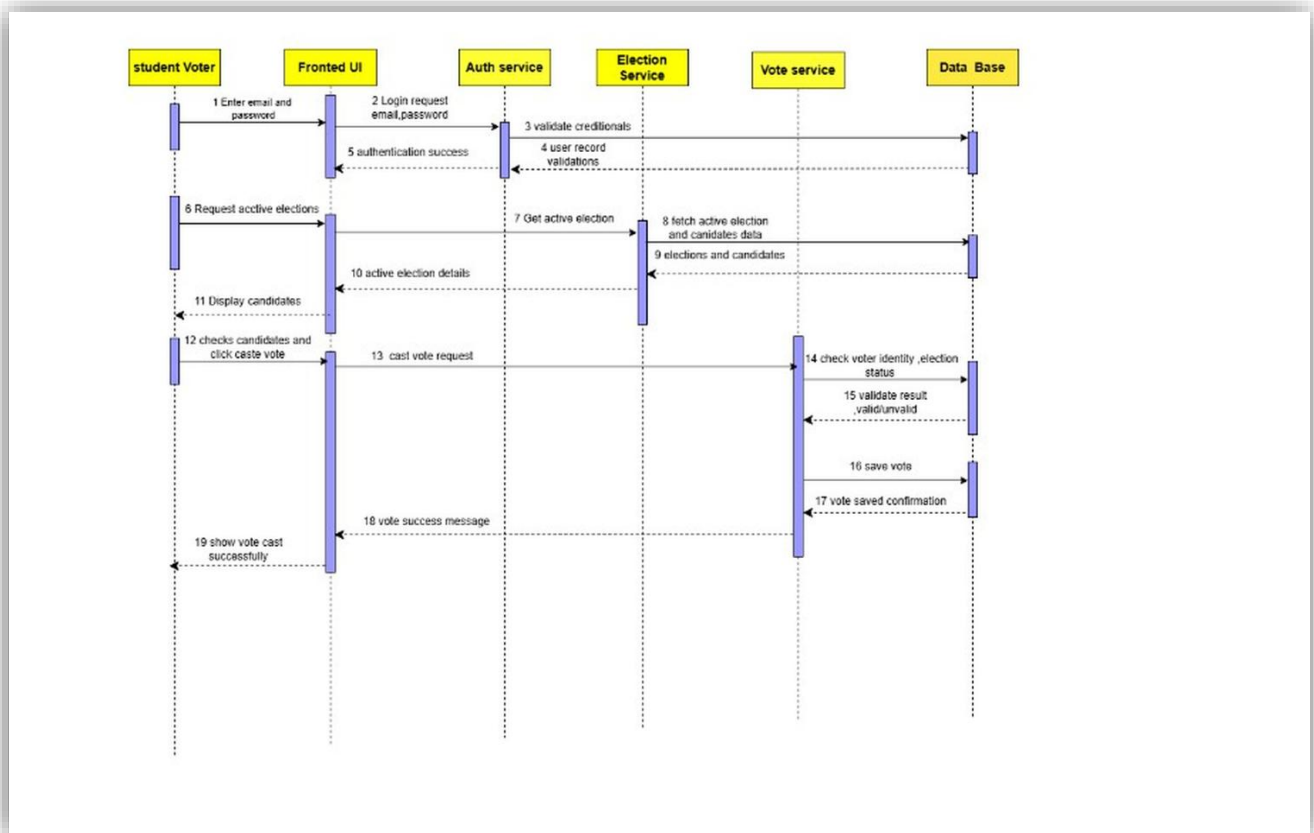


Figure 3: Sequence flow for login, election loading, voting, database storage, and result fetching.

## Activity Diagram

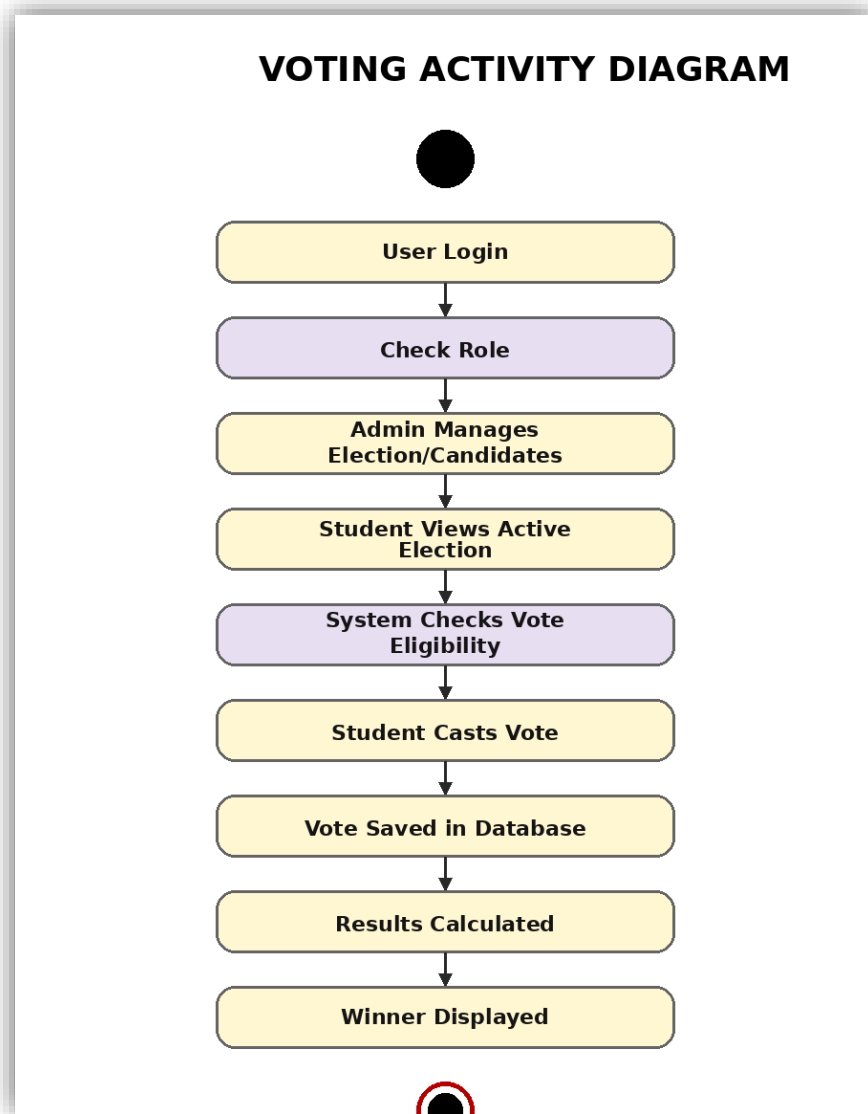


Figure 4: Activity flow of voting process from login to winner display.

## Testing Results and System Verification

The following screenshot placeholders are included for final proof of system execution. Replace each placeholder with actual screenshots from the working project before submission if required.

### Admin Dashboard Verification

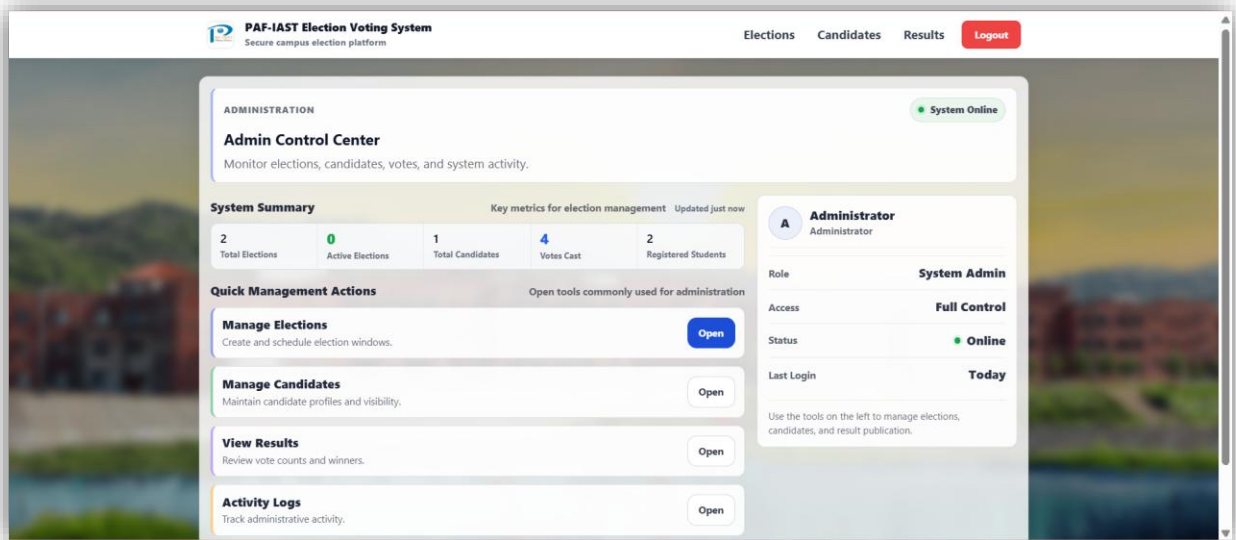


Figure: Admin Dashboard Verification confirms that the implemented system feature is working as expected.

## Student Voting Page Verification

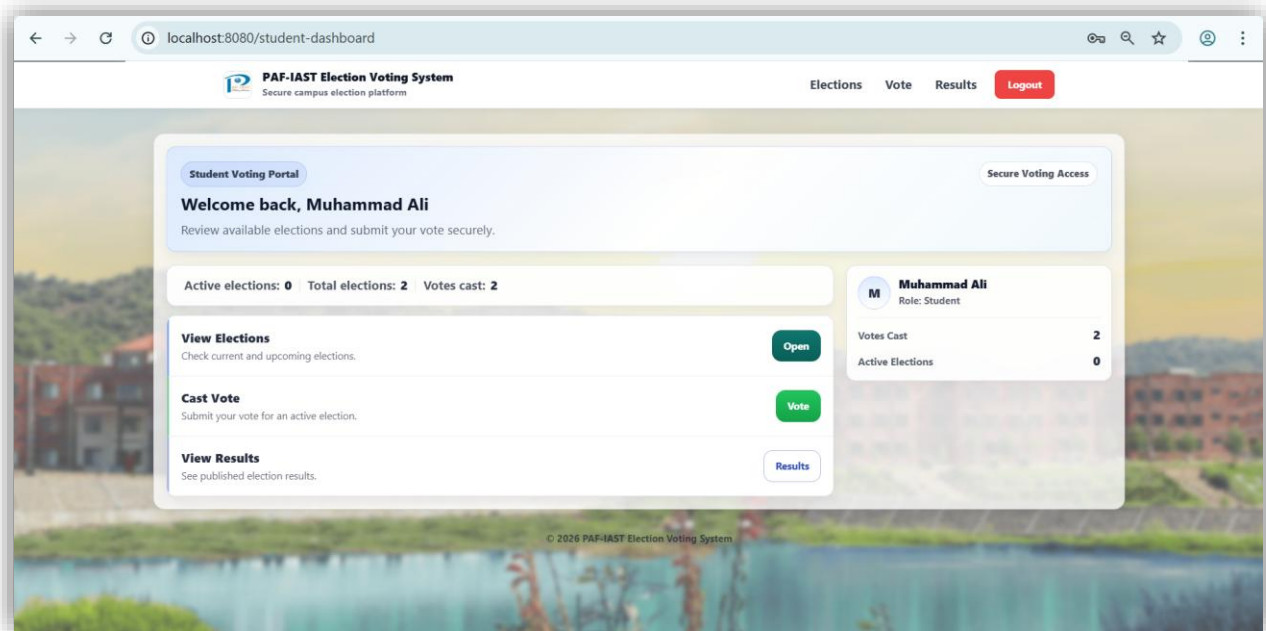
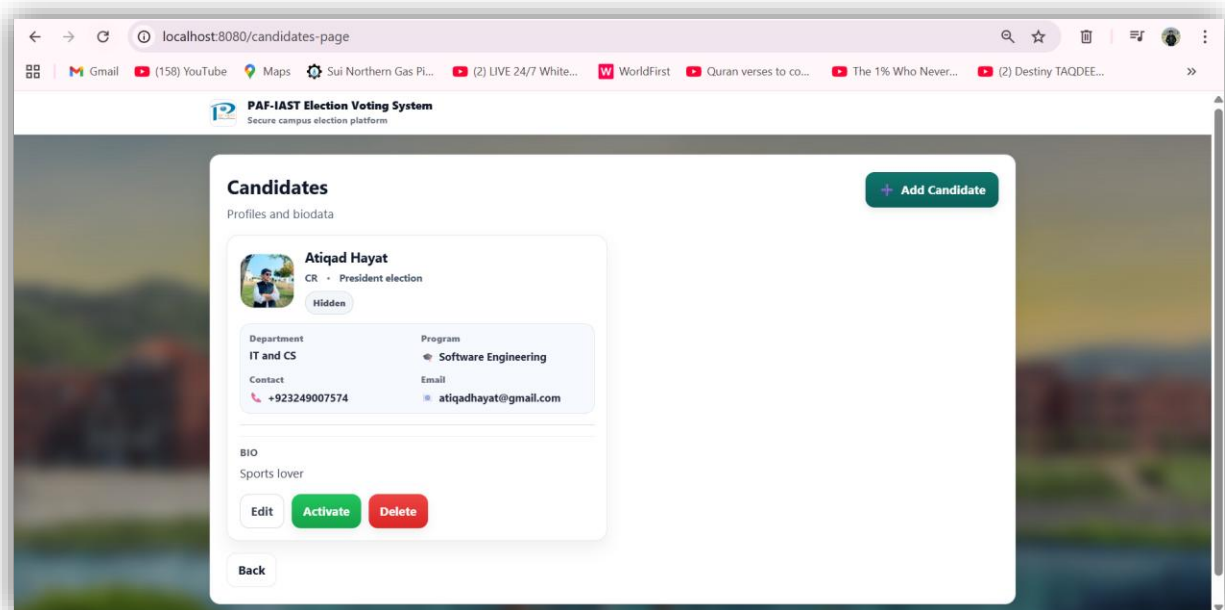
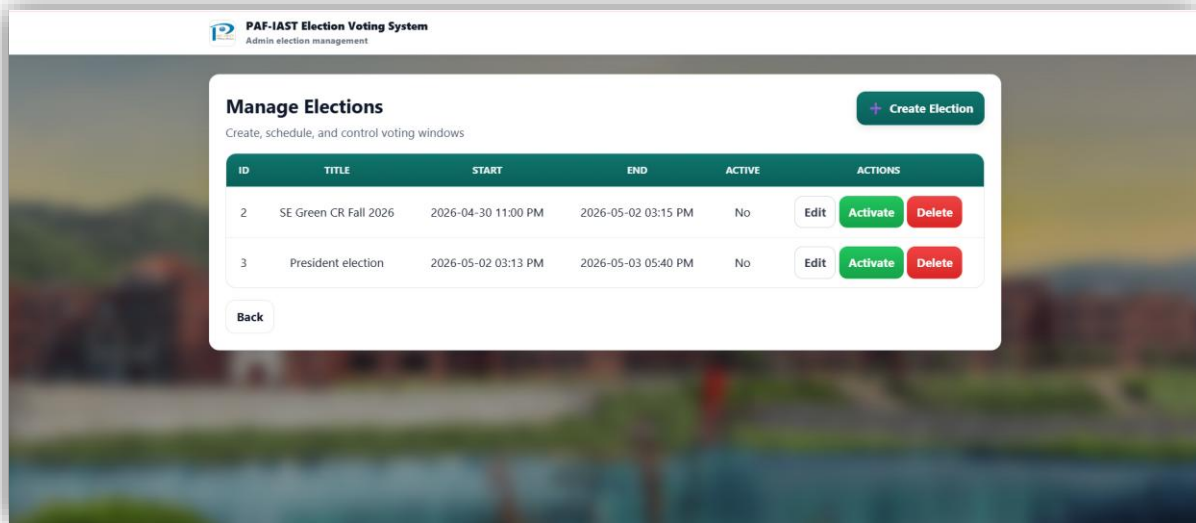


Figure: Student Voting Page Verification confirms that the implemented system feature is working as expected.

## Candidate Management Testing



## Election Management Testing



## Result and Winner Page Verification

